

Hybridization of Cartesian Genetic Programming and Differential Evolution for Generating Classifiers based on Neural Networks

Johnathan M. Melo Neto*, Heder S. Bernardino*, and Helio J.C. Barbosa*[†]

*Universidade Federal de Juiz de Fora, Juiz de Fora, MG, Brazil

[†]Laboratório Nacional de Computação Científica, Petrópolis, RJ, Brazil

jmmn.mg@gmail.com, heder@ice.ufjf.br, hcbm@lncc.br

Abstract—Despite the significance of Artificial Neural Networks (ANNs) in practical situations and the several works available in the literature, to adjust its parameters remains as a current problem. Hence, the advent of methods to assist users during this modeling is relevant. Three hybrid techniques based on Cartesian Genetic Programming (CGP) and Differential Evolution (DE) are proposed here for the construction of ANNs. The developed methods carry out an uncoupled evolution of the topology (using CGP) and the weights (using DE). The ANNs are evolved for classification problems, and seven benchmark datasets are used in the computational experiments. Results show the superiority of the proposed methods when compared to other techniques from the literature.

I. INTRODUCTION

The combination of Evolutionary Algorithms (EAs) and Artificial Neural Networks (ANNs), termed NeuroEvolution (NE), is a subfield of Machine Learning which applies EAs to the training of ANNs [1]. Some advantages of NE are the capability of manipulating the ANN topology, the suitability for supervised and reinforcement learning applications, and the ability to handle non-differentiable objective functions [1]. However, the training time is remarkably larger than that of other methods, such as backpropagation (BP).

Several works can be found in the literature where EAs are used to optimize ANNs [1]–[6]. Oong *et al.* [3] used an adaptive scheme to balance the global and local search in the evolutionary process, and a concurrent evolution of ANNs topology and weights. The approach, called HEANN, adapts the mutation probability and the step size of the weight perturbation. HEANN was tested on four benchmark functions and on real-world classification problems, showing superiority when compared with other algorithms, such as MGNN [7], NN-SAGP [8], and EPNet [9]. Although HEANN has many user-specified parameters, according to their authors the technique is not very sensitive to them. Garro *et al.* [5] introduced a methodology to automatically design an ANN based on the Differential Evolution (DE) algorithm. The method is capable of finding the topology, the synaptic weights, and the activation functions to solve classification problems. It was compared to BP and Particle Swarm Optimization based algorithms (PSO, 2GPSO, and MPSO), presenting better results when compared to PSO approaches, and a performance similar to that obtained when using BP.

Khan *et al.* [2] developed a method called CGPANN based on the application of Cartesian Genetic Programming (CGP) to the training of ANNs, optimizing the topology, the weights, and the transfer function of each node. The ANNs are encoded and evolved using a representation adapted from the CGP. That approach has been applied to pole balancing control problems [2], [10], to the forecasting of client requests in cloud data centers [11], and to a wide range of classification tasks [12], [13]. These works have shown that CGPANN is a highly competitive NE method, stand among competitive algorithms, such as NEAT, ESP, SANE, CoSyne, and others [2].

Nevertheless, a broader analysis performed by Turner [1] has shown a poor effectiveness of CGPANN in the domain of classification, indicating that the method is not recommended to deal with such problems. One of the possible explanations for this unsatisfactory performance is the use of CGP to evolve the connection weight genes, as its genotype was originally designed to be composed only of integer values [14]. Furthermore, the simultaneous optimization of topology and weights may lead to suboptimal solutions, as the entire configuration changes as a whole, without attempting to find a set of weights that best fits a particular topology.

To circumvent this issue, a hybrid approach combining both CGP and DE is proposed here. The developed method focuses on the uncoupled evolution of topology and weights to generate ANNs, where CGP is responsible for the topology evolution, and DE handles the weights learning. Such hybridization aims at providing better classifiers, as it relies on the use of a more appropriate algorithm for each type of task; CGP is suited to evolve the topology of graphs (such as ANNs) while DE was conceived to solve optimization problems in continuous search spaces [15].

Three methods based on this hybrid strategy were developed and compared with CGPANN and Gradient Descent BP in order to study the performance of the proposed methods when solving classification problems. One variant applies CGP and DE in an interleaved way, where DE optimizes the weights of the best ANN of each CGP generation. The other two variants execute DE only after the termination of CGP: one selects the best ANN from the DE population –with respect to the training data– whereas the other picks the best ANN regarding the validation data.

The remaining of this paper is organized as follows. Section II provides the basic concepts of CGP, CGPANN, and DE. Section III details the proposed techniques to evolve ANNs using the hybrid approach. Section IV presents the results and discussions of the computational experiments performed. Finally, Section V provides some concluding remarks.

II. EVOLUTIONARY TECHNIQUES

This section summarizes the basic concepts of the evolutionary algorithms adopted in the hybrid methods: Cartesian Genetic Programming (CGP), an extension of CGP when applied to ANNs (called CGPANN), and Differential Evolution.

A. Cartesian Genetic Programming

Cartesian Genetic Programming (CGP), firstly introduced in [14], is a type of Genetic Programming (GP) where each genotype depicts a directed acyclic graph, composed by function genes, connection genes, and output genes, as illustrated in Fig. 1. The function genes describe the functions of each node by using a look-up-table; e.g. 0 represents the addition function. The connection genes define from where each node obtains its inputs. In this work, nodes are allowed to connect to any other node on their left. Finally, the output genes determine which nodes are the program outputs. The nodes that do not contribute to the final program output are called inactive.

In its standard version, CGP uses an elitist ($\mu + \lambda$) Evolution Strategy (ES) [16], coupled with a probabilistic or point mutation. The initial population is usually randomly generated by assigning each gene to a random valid value. The best μ chromosomes are selected for the next generation, and λ mutated copies are created. The probabilistic mutation, which is used in this work, changes each gene in the genotype to a random valid value with a given probability. The basic CGP parameters are: μ , λ , the mutation rate, the maximum number of nodes, the number of inputs per node (arity), and the function set. Also, a stop criterion must be defined, such as a maximum number of generations. Some advantages of CGP are the benefits of neutral genetic drift, the natural suitability to Multiple-Input Multiple-Output (MIMO) tasks, and the absence of program bloat [1]. More information about CGP can be found in [1], [14], [17].

B. Cartesian Genetic Programming of Artificial Neural Networks

The Cartesian Genetic Programming of Artificial Neural Networks (CGPANN), originally proposed in [2], is an adapted version of CGP dedicated to the training of ANNs. CGPANN extends the standard CGP simply by the inclusion of connection weight genes for every connection gene in the chromosome. Each genotype is represented as an array of integers (describing the transfer functions and the connections between nodes), and floating-point values (describing the weights of each connection), as shown in Fig. 2. Thus, besides the topology and the transfer functions, mutation is also applied to the connection weight genes, changing a given

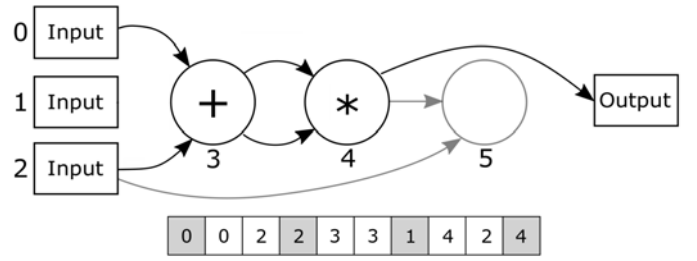


Figure 1. Example of a CGP graph with three inputs, three nodes and one output. Node 5 and its connections are inactive. The corresponding chromosome is illustrated below the graph, where the first three gray boxes represent the transfer functions of the respective nodes (0: add, 2: multiply, 1: subtract), the white boxes represent the connection genes of each node, and the last gray box refers to the output connection. Adapted from [1].

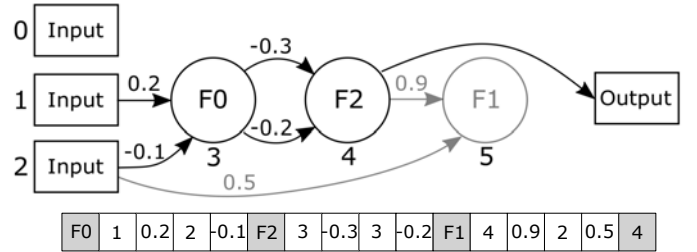


Figure 2. Example of a CGPANN graph with three inputs, three nodes and one output. Node 5 and its connections are inactive. The corresponding chromosome is illustrated below the graph, where $F0$, $F1$, and $F2$ (in gray) represent the transfer functions of the respective nodes, the white boxes represent the connection genes and their corresponding weights, and the last gray box refers to the output connection. Note that the only difference to CGP is the connection weight genes for each connection. Adapted from [1].

weight to a random value from an uniformly distributed range. Some advantages of this method are the natural suitability to describe neural networks, the non-local search of topology, and all the original benefits established by CGP theory [1]. Additional details of the CGPANN structure can be found in [1,2].

CGPANN also uses an elitist ($\mu + \lambda$)-ES to carry out the evolutionary process along the generations. Its pseudo-code is described in Algorithm 1. In all the experiments considered here, the dataset is partitioned in three distinct sets: training (TRN), validation (VLD), and testing (TST). The accuracy in the training set classification is used as the fitness of each chromosome, i.e., the greater the proportion of correctly classified instances, the better the solution. The validation set is used as an auxiliary set to select a potentially good ANN among the population in order to avoid overfitting. The ANN selected by the methods is evaluated by the testing set in order to assess its performance.

The ANNs modeled by CGPANN contains one output per class. Thus, the technique is capable of finding model for multiclass classification problems. Regarding the classification scheme, the output node that presents the largest value defines the class of the instance, e.g., if the values of the 3 output nodes of a chromosome are $O_1 = 0.25$, $O_2 = 0.34$, and $O_3 = 0.09$, then the given instance is labeled as class 2, as

O_2 presents the highest value.

The selection mechanism is performed on the whole population, and the mutation scheme is applied to both topology and weights (lines 13–14). During the process, the network that presents the best validation set accuracy is stored (lines 9–11), and at the end, the stored net accuracy is assessed using the testing set (line 16).

Algorithm 1 CGPANN: Pseudo-code

```

1: Initialize parameters ( $\mu$ ,  $\lambda$ ,  $MaxGen$ , etc.)
2: Create  $\mu + \lambda$  different nets
3: Evaluate accuracy of the  $\mu$  parents (TRN)
4: Create net  $\alpha$  to store best net
5: for  $gen = 1$  to  $MaxGen$  do
6:   for  $i = 1$  to  $\lambda$  do
7:     Evaluate accuracy of net  $i$  (TRN)
8:     Evaluate accuracy of net  $i$  (VLD)
9:     if Accuracy of net  $i \geq$  Accuracy of  $\alpha$  (VLD) then
10:      Net  $\alpha \leftarrow$  Net  $i$ 
11:   end if
12: end for
13: Select the  $\mu$  best nets among the  $\mu + \lambda$  for the next generation (TRN)
14: Create  $\lambda$  copies of the  $\mu$  parents selected & apply topology/weight mutations
15: end for
16: Evaluate accuracy of net  $\alpha$  (TST)

```

It can be noted that several data evaluations are executed throughout evolution. In order to provide a fair comparison with other algorithms, this paper introduces a metric called Evaluation Effort (EE). In EE, one training set evaluation is counted as an “unitary effort”, and one validation set evaluation is counted as a “ p effort”, in which p is the ratio between the sizes of the validation and training sets. Eq. 1 describes the EE used by CGPANN, wherein the terms are explicitly separated to highlight the evaluation effort of each set. In our experiments, cross-validation is adopted where the validation and training sets are formed by, respectively, 2 and 7 folds, as explained in Section IV-A. As a result, $p = 2/7$ here.

$$EE_{CGPANN} = MaxGen \cdot \lambda + \frac{2}{7} \cdot MaxGen \cdot \lambda \quad (1)$$

C. Differential Evolution

The Differential Evolution (DE) algorithm was first introduced in [18], and since then it has being widely applied to many optimization problems due to its good performance in continuous search spaces. It has few parameters and it converges faster than other evolutionary approaches [5]. DE is classified as a floating-point encoded evolutionary algorithm, i.e., the individuals’ representation is given by vectors of real numbers. A basic version of the method, called DE/rand/1/bin, is adopted here. In this variant, a donor vector v_i is generated by

$$v_i^{(j+1)} = x_{r_3}^{(j)} + F \cdot (x_{r_1}^{(j)} - x_{r_2}^{(j)}), \quad (2)$$

where j is the generation counter, $i = 1, \dots, Pop$, where Pop is the population size, F is a user defined parameter, and $x_{r_1}^{(j)}$, $x_{r_2}^{(j)}$ and $x_{r_3}^{(j)}$ are three individuals randomly selected from the current population. The crossover is applied with a rate CR and generates a trial vector $u_i^{(j+1)}$ by combining the donor vector $v_i^{(j+1)}$ with the target vector $x_i^{(j)}$ as

$$u_{ik}^{(j+1)} = \begin{cases} v_{ik}^{(j+1)}, & \text{if } r_4 \leq CR \text{ or } r_{id} = k \\ x_{ik}^{(j)}, & \text{otherwise,} \end{cases} \quad (3)$$

where r_4 is value randomly generated in $[0, 1)$, $k = 1, \dots, n$, r_{id} is an integer value randomly sampled from $\{1, \dots, n\}$, and n is the dimension of the optimization problem.

After this process, the fitness of the new element is compared to the target’s one, keeping the best solution in the population. This variant of DE is utilized throughout this work. For further information on the DE algorithm, see [15].

III. THE PROPOSED HYBRID METHODS

Given that CGP was originally developed to deal with discrete optimization problems, such as digital circuits [1], the CGPANN extension applied in the weight space of the neural networks may lead to suboptimal results. As DE is more suitable than CGP to handle optimization problems in continuous search spaces, the present study brings in a novel hybrid approach in which CGP and DE are used together to evolve classifiers based on neural networks. In this proposal, CGP is in charge of the topology evolution, whereas DE is dedicated to the evolution of the weights. To the best of the authors’ knowledge, this is the first time that a combination of both techniques is applied to create artificial neural networks. The three new methods based on CGP and DE are discussed hereafter.

A. CGPDE-IN

CGPDE-IN is a hybrid model that combines CGP and DE in an interleaved manner, as described in Algorithm 2. The $(\mu + \lambda)$ -ES is employed, which means the same CGPANN selection mechanism is used, however, mutation is applied solely on the topology, i.e., the connections between nodes are changed, but their weights remain the same (lines 15–16). On the other hand, in every generation, the best ANN from the offspring population is picked, and DE evolves its connection weight genes, keeping the topology fixed (lines 6–10). The initial population of DE is formed by the picked ANN and the other $Pop - 1$ individuals, which are generated using the same topology, but with different connection weights, varying each weight to a random value from an evenly distributed range. After $Iter$ iterations, DE returns the solution that presents the highest accuracy with respect to the training set.

Along the generations, and based on the validation set accuracy, the best chromosome among all processed by DE is stored (lines 11–14), and in the end it is utilized to classify the testing set (line 18).

As one can notice, DE periodically freezes a topology and concentrates its effort on adjusting the weights, whereas the

Algorithm 2 CGPDE-IN: Pseudo-code

```
1: Initialize parameters ( $\mu$ ,  $\lambda$ ,  $MaxGen$ , CR, F, etc.)
2: Create  $\mu + \lambda$  different nets
3: Evaluate accuracy of the  $\mu$  parents (TRN)
4: Create net  $\alpha$  to store best net
5: for  $gen = 1$  to  $MaxGen$  do
6:   for  $i = 1$  to  $\lambda$  do
7:     Evaluate accuracy of net  $i$  (TRN)
8:   end for
9:   Pick the best net among the  $\lambda$  (called  $\beta$ ) (TRN)
10:  DE evolves the weights of net  $\beta$  (TRN)
11:  Evaluate accuracy of net  $\beta$  (VLD)
12:  if Accuracy of net  $\beta \geq$  Accuracy of  $\alpha$  (VLD) then
13:    Net  $\alpha \leftarrow$  Net  $\beta$ 
14:  end if
15:  Select the  $\mu$  best nets among the  $\mu + \lambda$  for the next
  generation (TRN)
16:  Create  $\lambda$  copies of the  $\mu$  parents selected & apply
  topology mutation
17: end for
18: Evaluate accuracy of net  $\alpha$  (TST)
```

original CGPANN simultaneously modify both. Therefore, it is expected that this hybrid approach provides better models to correctly classify unseen data, as it allows each solution to find the set of weights that best suits its topology. Finally, the Evaluation Effort of CGPDE-IN (EE_{IN}) is indicated in Eq. 4, where $MaxGen$ is the max number of generations, λ is the offspring population, $Iter$ is the maximum iterations of DE, and Pop is the population of DE.

$$EE_{IN} = MaxGen \cdot (\lambda + Iter \cdot Pop) + \frac{2}{7} \cdot MaxGen \quad (4)$$

B. CGPDE-OUT

Similarly to the former strategy, this second hybrid approach, named CGPDE-OUT, generates neural networks by combining both CGP and DE. However, they are used in a separate way here, as presented in Algorithm 3. Initially, CGP evolves the topology through $(\mu + \lambda)$ -ES (lines 13–14), and across generations, the best solution evaluated on the validation set is selected (lines 9–11). When CGP is concluded, the weights of the selected individual (the best ANN with respect to the accuracy calculated using the validation data) are optimized by DE (line 16). As already mentioned, the DE algorithm produces Pop individuals with the same topology but with different weights. Two policies are proposed for selecting the best ANN after the DE evolution (line 17):

- CGPDE-OUT-T: selects the network that presents the highest accuracy with respect to the training data.
- CGPDE-OUT-V: selects the network that presents the best accuracy with respect to the validation data.

The selected network is used to classify the testing data (line 18). The main motivation for the development of these policies is the ability of the CGP to generate a good topology. Only after finding a reasonable topology, is DE applied to optimize

the weights that fit a particular configuration. Considering the above, the expectation is that CGPDE-OUT-V leads to better results than CGPDE-OUT-T version, as the use of the validation set to select the final network may increase the generalization capacity. Still, the Evaluation Effort is the same in both of them (Eq. 5):

$$EE_{OUT} = MaxGen \cdot \lambda + \frac{2}{7} \cdot MaxGen \cdot \lambda + Iter \cdot Pop \quad (5)$$

It is important to stress that all developed algorithms use the very same genotype, differing only in how and when they are modified.

Algorithm 3 CGPDE-OUT: Pseudo-code

```
1: Initialize parameters ( $\mu$ ,  $\lambda$ ,  $MaxGen$ , CR, F, etc.)
2: Create  $\mu + \lambda$  different nets
3: Evaluate accuracy of the  $\mu$  parents (TRN)
4: Create net  $\alpha$  to store best net
5: for  $gen = 1$  to  $MaxGen$  do
6:   for  $i = 1$  to  $\lambda$  do
7:     Evaluate accuracy of net  $i$  (TRN)
8:     Evaluate accuracy of net  $i$  (VLD)
9:     if Accuracy of net  $i \geq$  Accuracy of  $\alpha$  (VLD) then
10:      Net  $\alpha \leftarrow$  Net  $i$ 
11:     end if
12:   end for
13:   Select the  $\mu$  best nets among the  $\mu + \lambda$  for the next
  generation (TRN)
14:   Create  $\lambda$  copies of the  $\mu$  parents selected & apply
  topology mutation
15: end for
16: DE evolves the weights of net  $\alpha$  (TRN)
17: Pick the net with higher accuracy among the DE popu-
  lation: TRN (CGPDE-OUT-T) or VLD (CGPDE-OUT-V)
18: Evaluate accuracy of the picked net (TST)
```

IV. COMPUTATIONAL EXPERIMENTS

A. Experimental Setup

The results obtained by the proposed hybrid techniques are compared to those found by CGPANN and Gradient Descent Backpropagation Feedforward Network (BP). The evolutionary approaches are implemented in C¹ and they are an extension of the CGPANN source-code available in [19]. The BP is implemented using MATLAB Neural Network Toolbox. The parameters used for the experiments are shown in Table I.

For a fair comparison, the maximum number of generations of CGP, the maximum number of iterations of DE, and the DE population size were chosen in such a way that the Evaluation Effort of the evolutionary algorithms have similar values (Eqs. 1,4,5). The other parameters were estimated empirically based on preliminary experiments, using [1] as an initial guidance. Moreover, the BP algorithm used the MATLAB

¹Source-code available in <https://github.com/johnathanmelo/cgpde-lib.git>

Table I
PARAMETERS USED BY THE EVOLUTIONARY ALGORITHMS

Parameter	CGPANN	CGPDE-IN	CGPDE-OUT
Max Generations	50000	64	40000
DE Max Iterations	-	400	2570
DE Population	-	10	20
F	-	0.70	0.70
CR	-	0.90	0.90
Mutation Scheme	probabilistic		
Mutation Rate	0.05		
Transfer Function	logistic sigmoid		
Evolutionary Strategy	(1+4)-ES		
Number of Nodes	500		
Node Arity	20		
Initial Weight Range	[-5, +5]		

Table II
DATASETS SUMMARY SELECTED FOR THE EXPERIMENTS

Dataset	Instances	Attributes	Classes
Glass Identification	214	9	6
Pima Indians Diabetes	768	8	2
Iris	150	4	3
Wisconsin Breast Cancer [21]	683	9	2
Ionosphere	351	34	2
Vertebral Column	310	6	3
Wine	178	13	3

default configuration, with 500 neurons in the hidden layer, and the initial weights are randomly initialized.

Seven datasets from the UCI Machine Learning Repository [20] were used for the experiments, and they are described in Table II. Each attribute is linearly normalized into the [0,1] range. The stratified 10-fold cross-validation is applied, i.e., the instances are shuffled and divided in 10 folds of approximately the same size, keeping the same class proportion in each fold. Initially, the fold #1 is reserved for the testing set, and the remaining 9 folds are randomly distributed: 7 folds for the training set and 2 folds for the validation set. In the next run, the fold #2 is reserved for the testing set, while the remaining ones are divided between training and validation sets. This procedure is repeated for all the 10 folds. The same sets are used by all the methods considered here. As the search techniques used here are stochastic, and in order to increase the number of results to analyze, the entire scheme is repeated three times. Thus, 30 independent runs are performed.

B. Results and Analysis

The data of the 30 testing set accuracies of the classifiers for each dataset is illustrated by the box plots in Fig. 3. It can be noted that the median values of the results obtained by the hybrid methods are higher than those found by BP and CGPANN. Also, the variability of the data is more prominent in CGPANN, although in few cases other methods have presented both wider spread and interquartile range.

To further extend the analysis, Table III presents the mean and standard deviation of the testing set accuracies. As it can be seen, in five of the seven datasets used here, CGPDE-IN outperformed the other methods, presenting statistical evi-

Table III
TESTING SET ACCURACY: MEAN & STD. DEV. BOLDFACE VALUES INDICATE THE HIGHEST MEAN. STARRED VALUES INDICATE THE REJECTION OF THE NULL HYPOTHESIS AT THE 0.05 SIGNIFICANCE LEVEL AGAINST THE HIGHEST MEAN METHOD USING THE ONE-TAILED PAIRED-SAMPLE *t*-TEST.

Dataset	BP	CGPANN	CGPDE-IN	CGPDE-OUT-T	CGPDE-OUT-V
Glass	0.5812 (0.1085)	0.5731* (0.0937)	0.6198 (0.0769)	0.5977 (0.0874)	0.6058 (0.0923)
Diabetes	0.7427 (0.0496)	0.7361* (0.0580)	0.7549 (0.0409)	0.7379* (0.0461)	0.7454 (0.0475)
Iris	0.9355 (0.0510)	0.9178* (0.0693)	0.9489 (0.0516)	0.9444 (0.0556)	0.9467 (0.0537)
Cancer	0.9532* (0.0241)	0.9546* (0.0263)	0.9609 (0.0257)	0.9644 (0.0240)	0.9648 (0.0246)
Ionosphere	0.8785 (0.0477)	0.8433* (0.0550)	0.8927 (0.0435)	0.8936 (0.0444)	0.8927 (0.0402)
Vertebral	0.7882 (0.0654)	0.7151* (0.0749)	0.8204 (0.0798)	0.8097 (0.0716)	0.8000 (0.0755)
Wine	0.9402 (0.0582)	0.8578* (0.1072)	0.9533 (0.0444)	0.9045* (0.0765)	0.9138* (0.0598)

Table IV
p-VALUE OF THE ONE-TAILED PAIRED-SAMPLE *t*-TEST AGAINST CGPANN. BOLDFACE VALUES INDICATE A FAILURE TO REJECT THE NULL HYPOTHESIS AT THE 0.05 SIGNIFICANCE LEVEL.

Dataset	CGPDE-IN	CGPDE-OUT-T	CGPDE-OUT-V
Glass	0.0044	0.1020	0.0330
Diabetes	0.0478	0.4374	0.2298
Iris	0.0102	0.0314	0.0201
Cancer	0.0888	0.0310	0.0253
Ionosphere	<0.0001	<0.0001	<0.0001
Vertebral	<0.0001	<0.0001	<0.0001
Wine	<0.0001	0.0300	0.0095

dence that it is better than CGPANN, as well as CGPDE-OUT-T (Diabetes and Wine) and CGPDE-OUT-V (Wine), as the starred values indicate. Furthermore, for the Cancer dataset, CGPDE-OUT-V surpassed the mean accuracy of the other classifiers, and it is statistically superior to BP and CGPANN. For the Ionosphere dataset, CGPDE-OUT-T showed superior behavior, with statistical evidence that it is better than CGPANN. In summary, it can be concluded that the proposed hybrid evolutionary methods not only exhibited larger mean accuracy than BP and CGPANN for all datasets, but also they provided evidence of their superiority over the original CGPANN, as can be verified in Table IV. Comparing only the CGPDE-OUT versions, the use of validation data (CGPDE-OUT-V) showed better performance than using training data (CGPDE-OUT-T), as initially expected. Finally, the CGPDE-IN algorithm produced better results compared to the other hybrid versions, presenting a higher mean, and best statistical results for the majority of the datasets, suggesting that the interleaved evolution of topology and weights using CGP and DE generates better ANNs for classification problems and with good generalization capability.

A comparison in terms of the structures of the networks

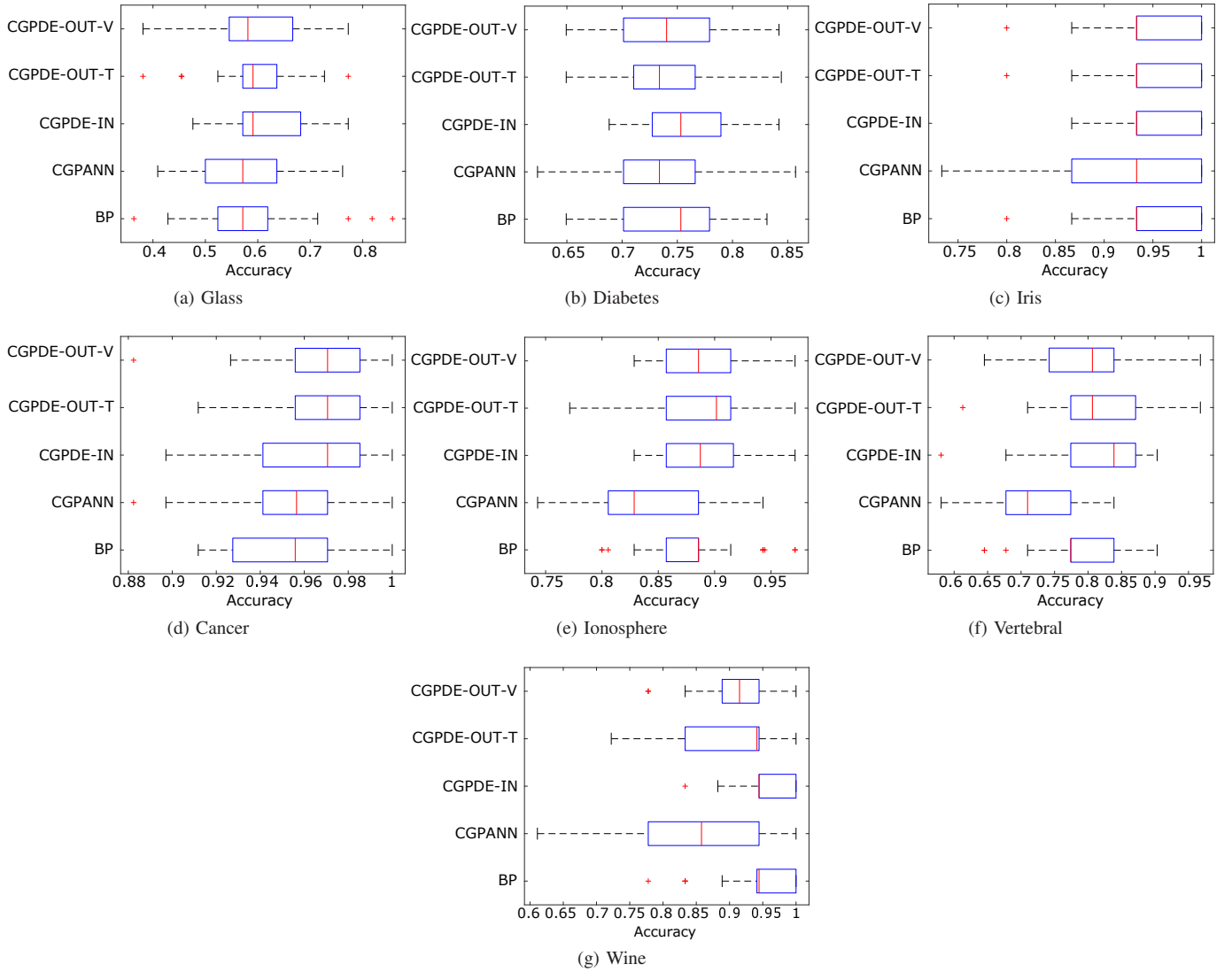


Figure 3. Box plots of the 30 testing set accuracies of each algorithm for the seven datasets.

Table V
NUMBER OF ACTIVE NODES OF THE FINAL NETWORKS: MEAN & STD. DEV. BOLDFACE VALUES INDICATE THE LOWEST MEANS.

Dataset	CGPANN	CGPDE-IN	CGPDE-OUT
Glass	375 (049)	370 (052)	372 (048)
Diabetes	269 (101)	306 (105)	303 (089)
Iris	316 (090)	328 (083)	321 (089)
Cancer	295 (096)	275 (112)	283 (094)
Ionosphere	252 (122)	300 (078)	261 (116)
Vertebral	309 (078)	323 (080)	320 (085)
Wine	315 (084)	302 (084)	272 (104)

is also performed. Table V shows the mean and standard deviation of the active nodes of the final networks generated by the evolutionary methods considered here. In general, the mean values are similar. In addition, the higher standard deviations indicate that the methods have produced a wide range of topologies. Although the hybrid methods and the CGPANN

have differed with respect to classification accuracy, their mean number of active nodes did not differ significantly, suggesting that the number of active nodes does not seem to directly influence their performance.

In order to analyze the performance of the evolutionary approaches when dealing with smaller sample sizes, the methods were tested on reduced datasets. We randomly picked a percentage of the instances of each dataset, keeping the same class proportion, and executed the same experimental setup previously described. The percentages used here are 25%, 50%, 75%, and 100% of the original datasets, and the testing set accuracy of each algorithm is illustrated in Fig. 4. In most of the scenarios, the classification accuracy tends to degrade when the sample size decreases. CGPDE-IN exhibited the smallest variation in the performance in most cases. Both CGPDE-OUT versions presented similar accuracy values when varying the amount of instances. Finally,

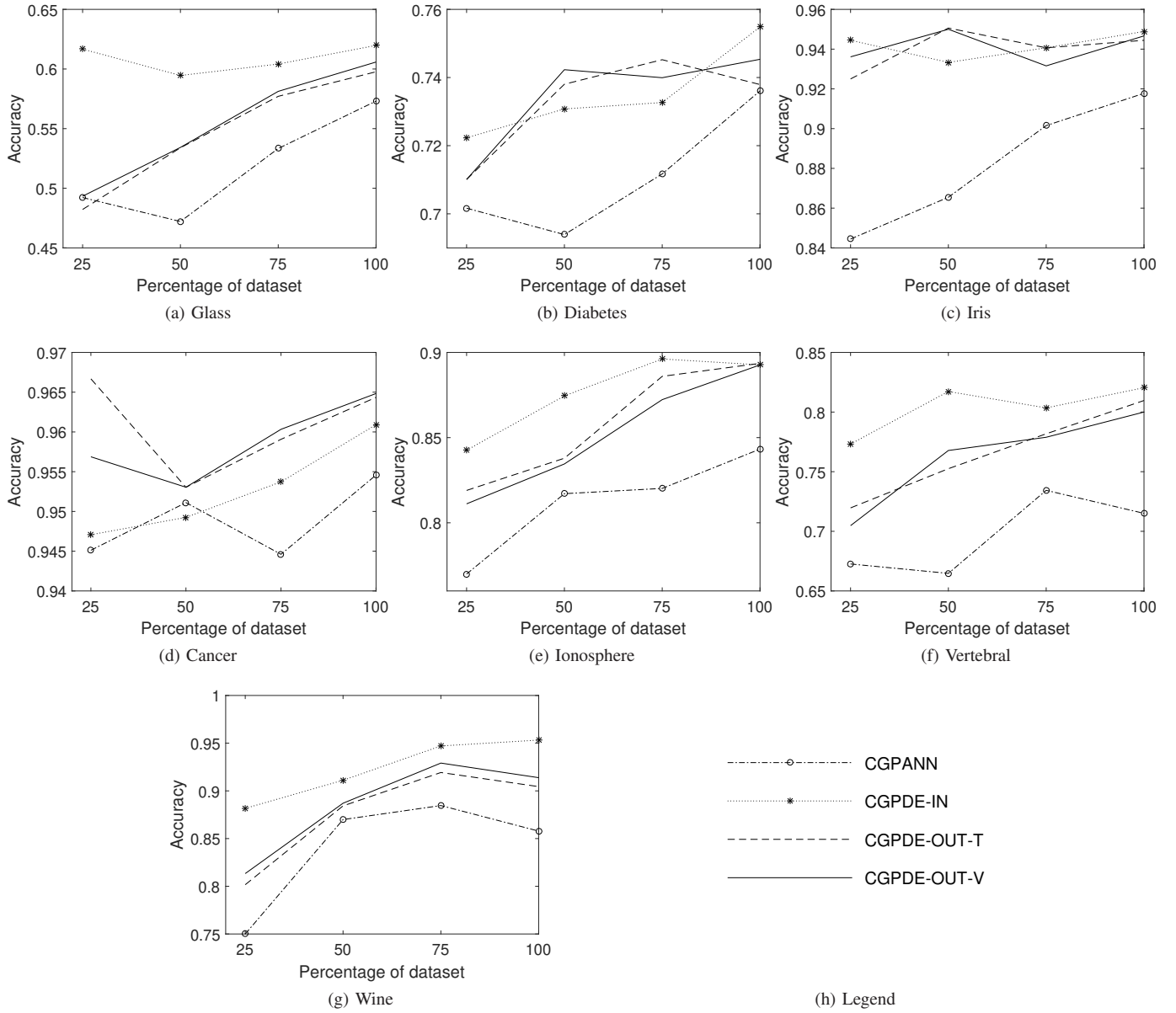


Figure 4. Testing set accuracy mean of each algorithm, using 25%, 50%, 75%, and 100% of the original datasets.

CGPANN presented the worst accuracy in all experiments.

V. CONCLUDING REMARKS

This paper has proposed three hybrid approaches which combine Cartesian Genetic Programming and Differential Evolution towards the generation of Artificial Neural Networks as an alternative to the original CGPANN and the algorithms commonly used in the literature for training ANNs. The main contribution here was the uncoupled evolution of topology and weights: the topology optimization is performed by CGP while DE finds the best weights for the connections. The CGPDE-IN version applies CGP and DE in an interleaved manner, where DE optimizes the weights of the best ANN of each CGP generation. The CGPDE-OUT versions execute DE only after the termination of CGP: CGPDE-OUT-T selects the best

ANN from the population of DE with respect to the training data, whereas CGPDE-OUT-V picks the best one regarding the validation data.

The proposed methods were applied to seven benchmark classification datasets and compared with CGPANN and BP. The experimental results suggest the superiority of the hybridization techniques, with statistical support that they produce better classifiers than CGPANN, especially the CGPDE-IN version. Also, CGPDE-IN has presented the best performance when dealing with reduced sample size, showing more robustness when compared to CGPANN. Moreover, a structural analysis was performed, and the number of active nodes does not differ significantly among the evolutionary methods.

It therefore appears conclusive that, due to the current good results, these hybrid propositions should be further studied in order to verify their classification capacity on more complex real-world datasets. Furthermore, a possible future research topic is the use of other objective functions, as the techniques proposed do not require their differentiability, and other metrics may be useful in handling imbalanced data classification problems.

ACKNOWLEDGMENT

The authors thank the reviewers for their comments, and the financial support provided by FAPEMIG (grants APQ-03414-15 and PCE-00439-18), CNPq (grant 310778/2013-1), PPGMC, and PPGCC.

REFERENCES

- [1] A. Turner, "Evolving artificial neural networks using cartesian genetic programming," Ph.D. dissertation, University of York, 2015.
- [2] M. M. Khan, G. M. Khan, and J. F. Miller, "Evolution of neural networks using cartesian genetic programming," in *IEEE Congress on Evolutionary Computation*, July 2010, pp. 1–8.
- [3] T. H. Oong and N. A. M. Isa, "Adaptive evolutionary artificial neural networks for pattern classification," *IEEE Transactions on Neural Networks*, vol. 22, no. 11, pp. 1823–1836, Nov 2011.
- [4] N. J. Radcliffe, "Genetic set recombination and its application to neural network topology optimisation," *Neural Computing & Applications*, vol. 1, no. 1, pp. 67–90, Mar 1993.
- [5] B. A. Garro, H. Sossa, and R. A. Vázquez, *Design of Artificial Neural Networks Using Differential Evolution Algorithm*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 201–208.
- [6] J. Ilonen, J.-K. Kamarainen, and J. Lampinen, "Differential evolution training algorithm for feed-forward neural networks," *Neural Processing Letters*, vol. 17, no. 1, pp. 93–105, Feb 2003.
- [7] P. P. Palmes, T. Hayasaka, and S. Usui, "Mutation-based genetic neural network," *IEEE Transactions on Neural Networks*, vol. 16, no. 3, pp. 587–600, May 2005.
- [8] J. Ang, K. Tan, and A. Al-Mamun, "Training neural networks for classification using growth probability-based evolution," *Neurocomputing*, vol. 71, no. 16, pp. 3493 – 3508, 2008, advances in Neural Information Processing (ICONIP 2006) / Brazilian Symposium on Neural Networks (SBRN 2006).
- [9] X. Yao and Y. Liu, "A new evolutionary system for evolving artificial neural networks," *IEEE Transactions on Neural Networks*, vol. 8, no. 3, pp. 694–713, May 1997.
- [10] M. M. Khan, G. M. Khan, and J. F. Miller, "Evolution of optimal anns for non-linear control problems using cartesian genetic programming," in *In Proceedings of IEEE International Conference in Artificial Intelligence*, 2010.
- [11] J. Ali, F. Zafari, G. M. Khan, and S. A. Mahmud, "Future clients' requests estimation for dynamic resource allocation in cloud data center using cgpann," in *2013 12th International Conference on Machine Learning and Applications*, vol. 2, Dec 2013, pp. 331–334.
- [12] A. M. Ahmad, G. M. Khan, S. A. Mahmud, and J. F. Miller, "Breast cancer detection using cartesian genetic programming evolved artificial neural networks," in *Proceedings of the 14th Annual Conference on Genetic and Evolutionary Computation*, ser. GECCO '12. New York, NY, USA: ACM, 2012, pp. 1031–1038.
- [13] A. M. Ahmad and G. M. Khan, "Bio-signal processing using cartesian genetic programming evolved artificial neural network (cgpann)," in *2012 10th International Conference on Frontiers of Information Technology*, Dec 2012, pp. 261–268.
- [14] J. F. Miller and P. Thomson, *Cartesian Genetic Programming*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2000, pp. 121–132.
- [15] S. Das and P. N. Suganthan, "Differential evolution: A survey of the state-of-the-art," *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 1, pp. 4–31, Feb 2011.
- [16] I. Rechenberg, "Evolutionsstrategie: Optimierung technischer systeme nach prinzipien der biologischen evolution," Ph.D. dissertation, TU Berlin, 1971.
- [17] J. F. Miller, *Cartesian Genetic Programming*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 17–34.
- [18] R. Storn and K. Price, "Differential evolution—a simple and efficient adaptive scheme for global optimization over continuous spaces," ICSI, USA, Tech. Rep. TR-95-012, 1995.
- [19] A. J. Turner and J. F. Miller, "Introducing a cross platform open source cartesian genetic programming library," *Genetic Programming and Evolvable Machines*, vol. 16, no. 1, pp. 83–91, Mar 2015.
- [20] M. Lichman, "UCI machine learning repository," 2013.
- [21] O. Mangasarian, R. Setiono, and W. H. Wolberg, "Pattern recognition via linear programming: Theory and application to medical diagnosis," 1990.